# Experiment 7

**Aim:** Implement the embedding wallet (Metamask) and transaction using Solidity

**Task to be performed:**

1. Set Up MetaMask:
2. Install MetaMask
3. Create or Import an Account:
4. Fund Your Wallet : Sepolia Testnet (0.5 ETH per day) / RSK Testnet (0.05 RBTC per day)
5. Connect the Sepolia Testnet / RSK Testnet to Remix IDE
6. Create a Simple Solidity Smart Contract based on the Mini Project chosen
7. Compile and Deploy the Smart Contract.
8. Check the transaction details on the RSK Explorer
9. Interact with the smart contract

**Theory:**

**MetaMask** is a widely used cryptocurrency wallet and browser extension that serves as a gateway to blockchain-based applications. It enables users to securely store, manage, and transfer digital assets such as Ether (ETH) and other tokens built on blockchain networks. Beyond just a wallet, MetaMask allows seamless interaction with decentralized applications (DApps) directly through a web browser, eliminating the need for running a full blockchain node.

MetaMask functions as a bridge between the user and blockchain networks like Ethereum and other compatible ecosystems. It injects a Web3 interface into the browser, enabling websites to communicate with the blockchain. This makes it especially useful for developers and users working with smart contracts, decentralized finance (DeFi) platforms, NFTs, and more.

One of the key strengths of MetaMask is its emphasis on security and user control. Private keys are stored locally on the user's device and are encrypted with a password, ensuring that only the user has access to their funds. It also provides a seed phrase for account recovery, which must be kept secure at all times.

**Features of MetaMask**

- Easy creation and management of multiple blockchain accounts within a single interface
- Secure storage of private keys and sensitive credentials
- Support for multiple networks, including Ethereum mainnet and custom networks
- Integration with development tools like Remix IDE for deploying and testing smart contracts
- Ability to connect and interact with a wide range of DApps such as DeFi platforms and NFT marketplaces
- Built-in token swapping and transaction tracking features

**Test Networks (Testnets)**

A **Testnet** is a blockchain network specifically designed for testing and development purposes. It replicates the functionality of a real blockchain (mainnet) but uses valueless cryptocurrency, allowing developers to experiment without financial risk. Testnets are essential for building, debugging, and validating smart contracts and decentralized applications before deploying them to a live environment.

Popular Ethereum-compatible test networks include Sepolia Testnet, Linea Testnet, and RSK Testnet. These networks provide developers with free test Ether (via faucets) to simulate real transactions, deploy contracts, and evaluate performance.

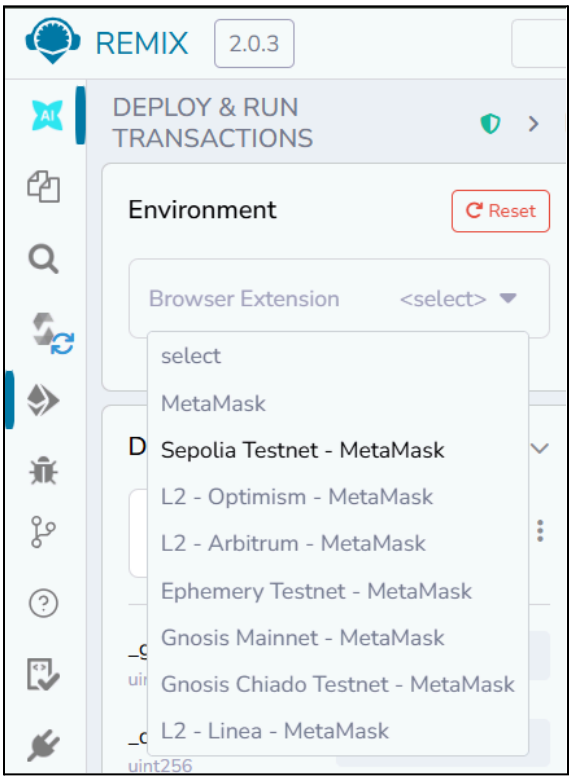
Testnets closely mirror real-world blockchain conditions, including transaction fees, network latency, and execution behavior. This ensures that applications are thoroughly tested and optimized before being launched on the mainnet, where real assets are involved.

**Advantages of Testnets**

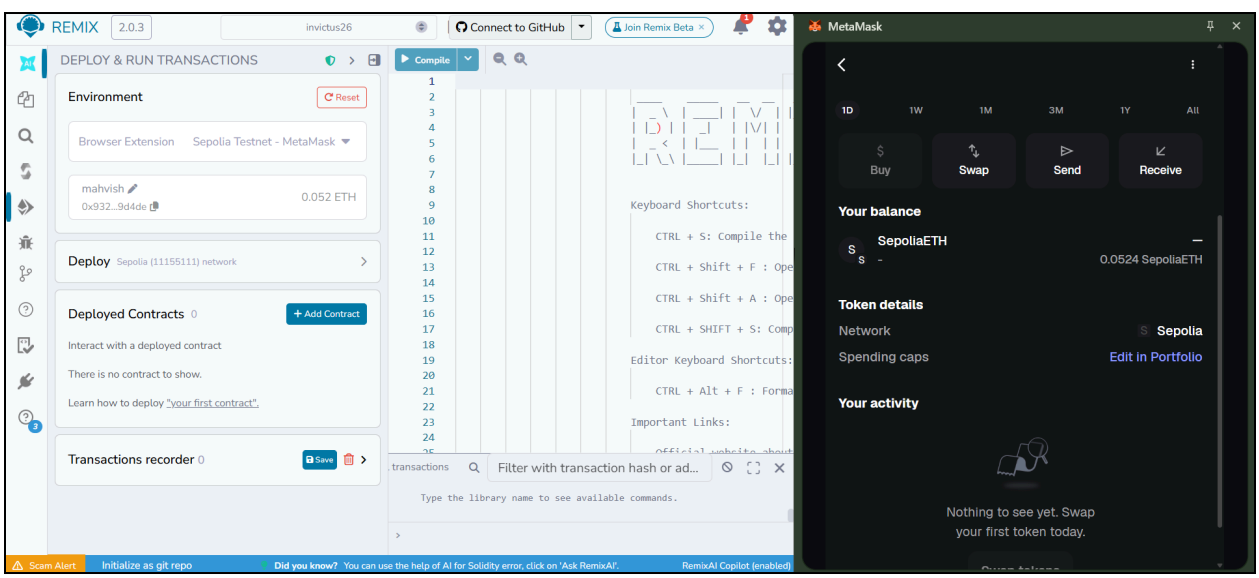
- Eliminates the risk of losing real cryptocurrency during experimentation
- Provides a safe and controlled environment for developers and learners
- Enables thorough testing of smart contracts, DApps, and transactions
- Helps identify bugs, vulnerabilities, and performance issues early
- Allows developers to gain hands-on experience with blockchain systems

## Implementation

Step 1: Select metamask sepolia testnet in environment




Initial setup:



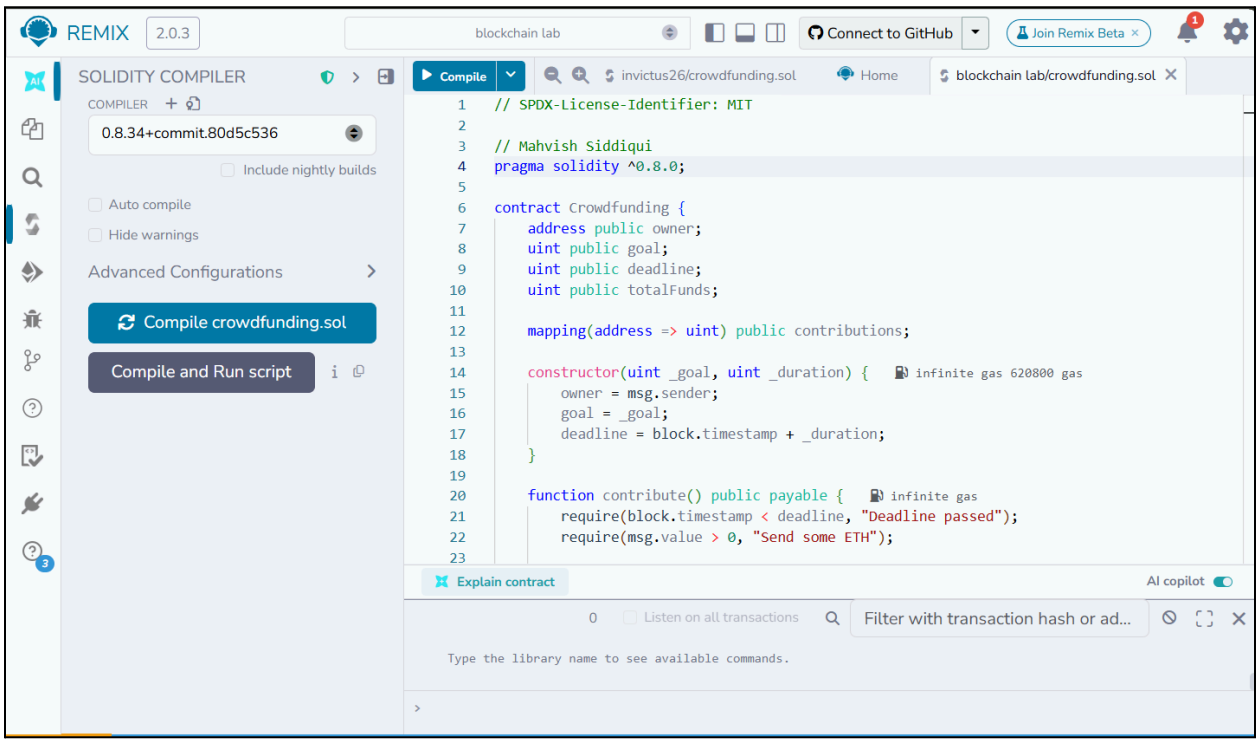
**Step 2:** Create a contract for a crowdfunding platform

Code:

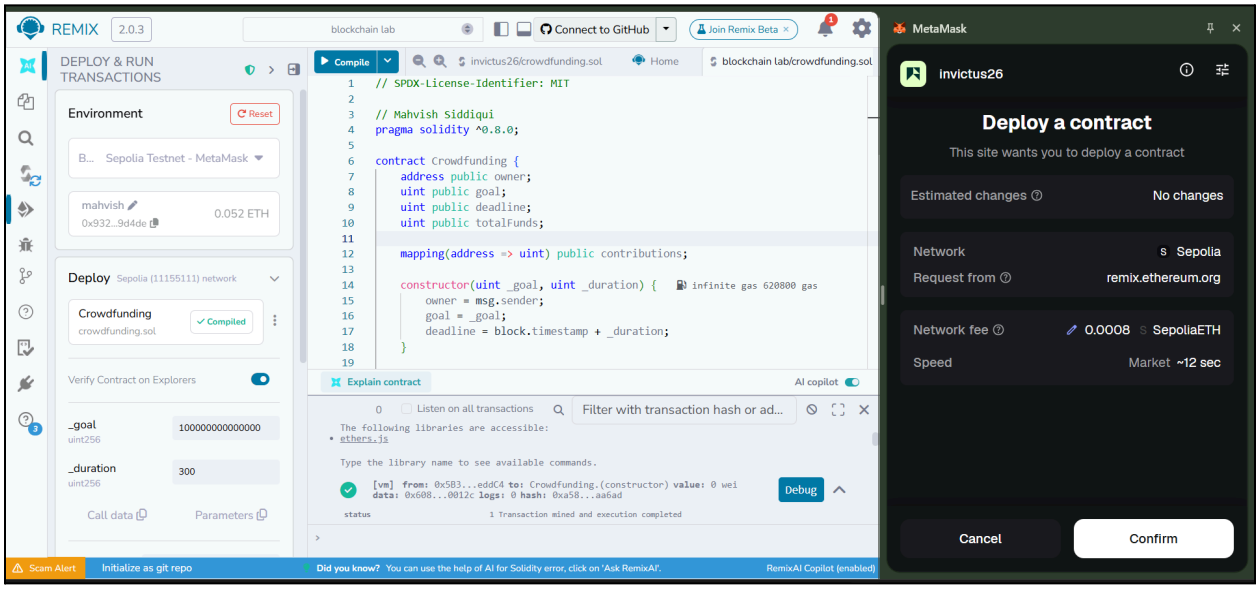
```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;

contract Crowdfunding {
    address public owner;
    uint public goal;
    uint public deadline;
    uint public totalFunds;

    mapping(address => uint) public contributions;

    constructor(uint _goal, uint _duration) {
        owner = msg.sender;
        goal = _goal;
        deadline = block.timestamp + _duration;
    }

    function contribute() public payable {
        require(block.timestamp < deadline, "Deadline passed");
        require(msg.value > 0, "Send some ETH");

        contributions[msg.sender] += msg.value;
        totalFunds += msg.value;
    }
    function withdrawFunds() public {
        require(msg.sender == owner, "Only owner");
        require(totalFunds >= goal, "Goal not reached");

        (bool success, ) = payable(owner).call{value: totalFunds}("");
        require(success, "Transfer failed");
    }

    function refund() public {
        require(block.timestamp > deadline, "Not ended");
        require(totalFunds < goal, "Goal was reached");

        uint amount = contributions[msg.sender];
        require(amount > 0, "No contribution");
```

```
        contributions[msg.sender] = 0;

        (bool success, ) = payable(msg.sender).call{value: amount}("");
        require(success, "Refund failed");
    }

    function getBalance() public view returns (uint) {
        return address(this).balance;
    }
}
```

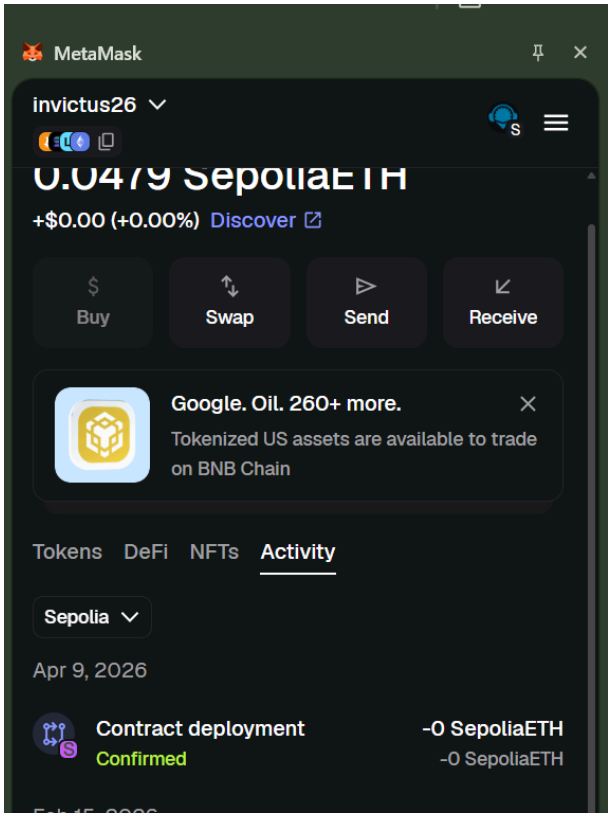
**Step 3**: save it as **crowdfunding.sol** and compile it



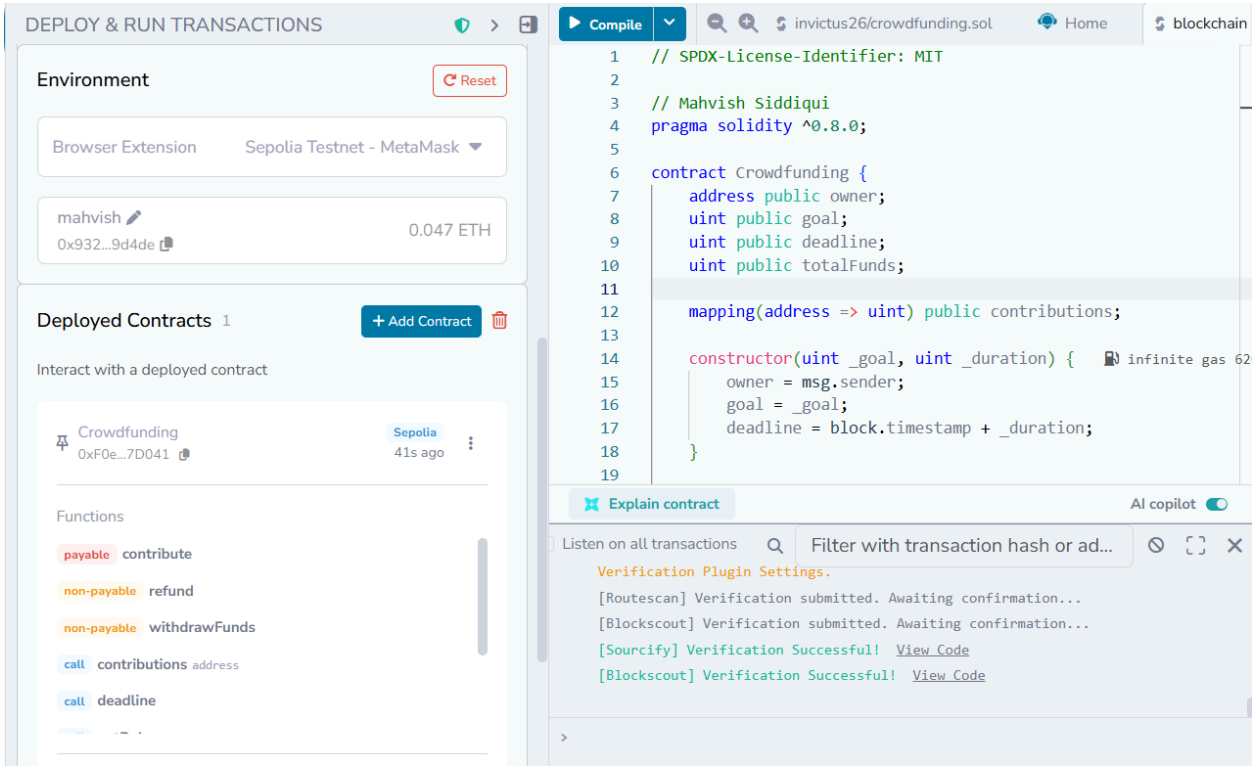
**Step 4:** deploy the contract with an initial goal and time limit. Confirm the transaction from metamask



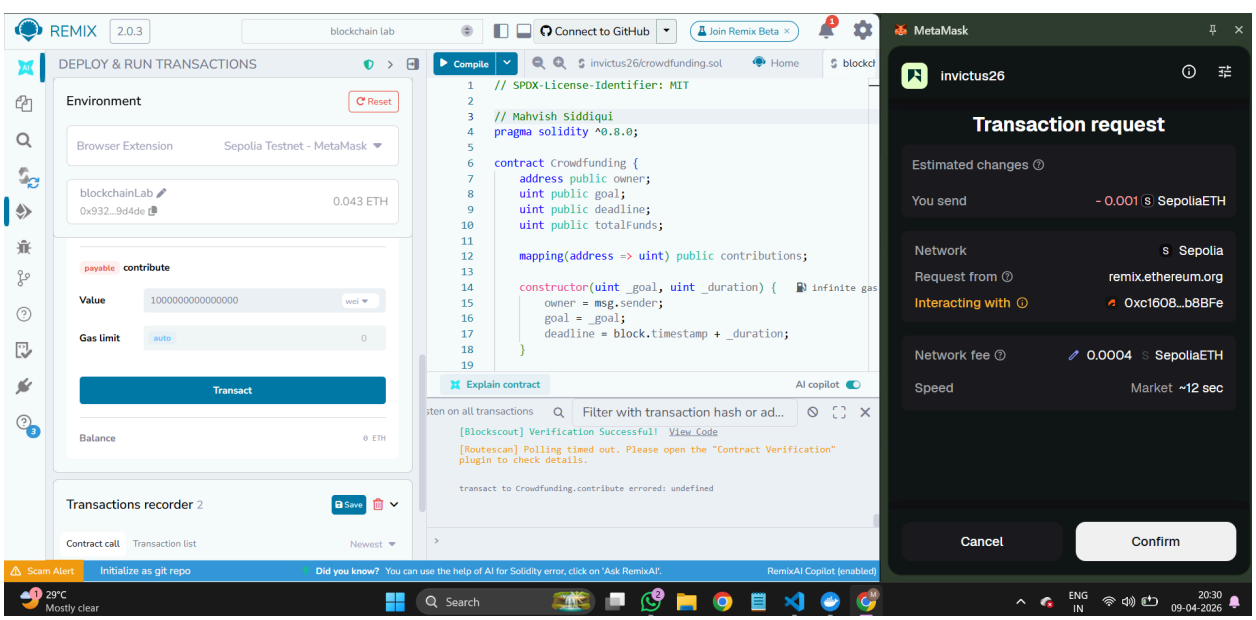

**After deployment you can see:** transaction recorded in metamask

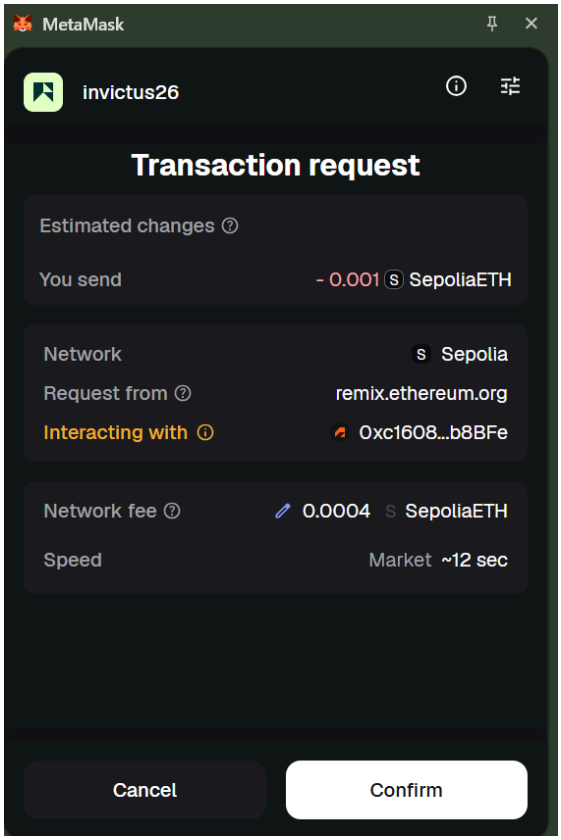


**Step 5:** After deployment in remix ide, we can see the deployed contract and its functions

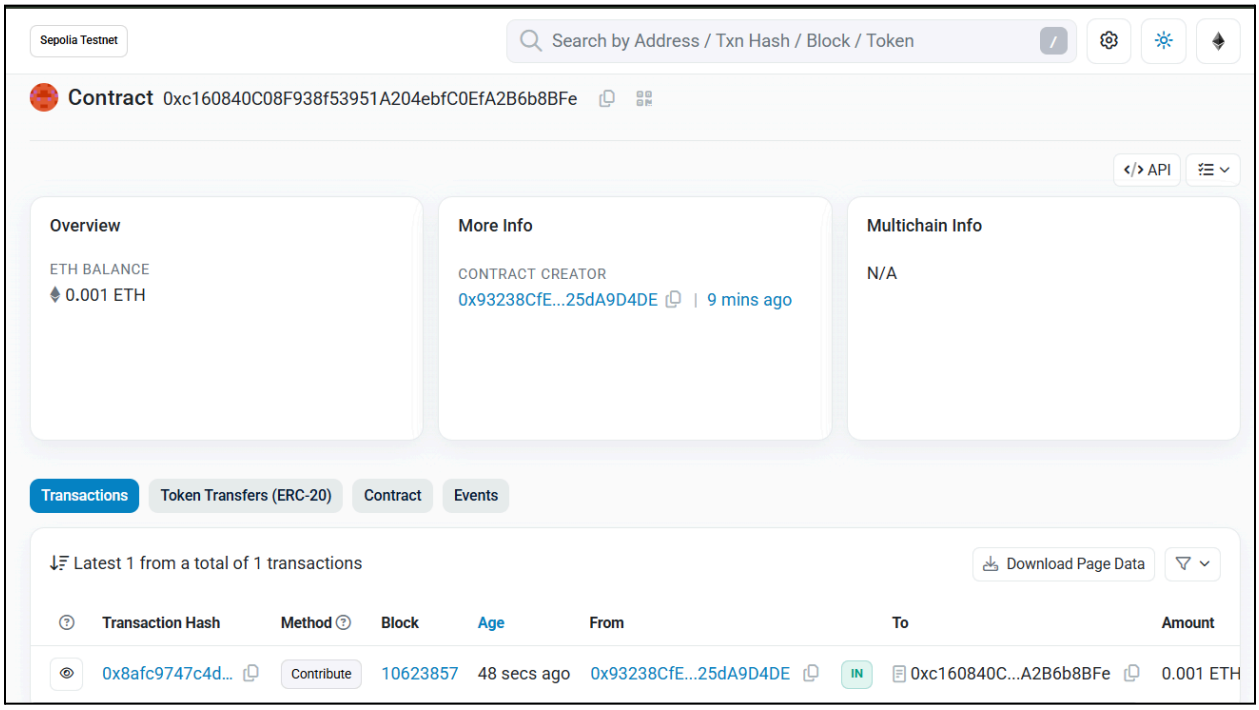



**Step 6:** Add a contribution of 0.01 ETH = 1000000000000000 wei. Transaction request appears in metamask. Click on confirm

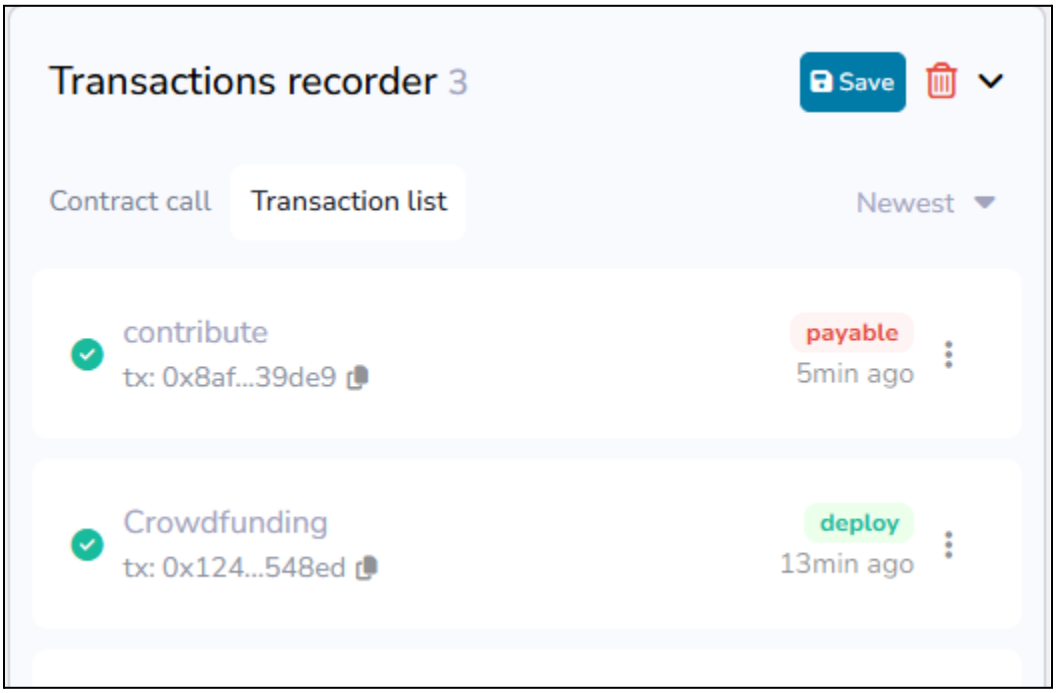




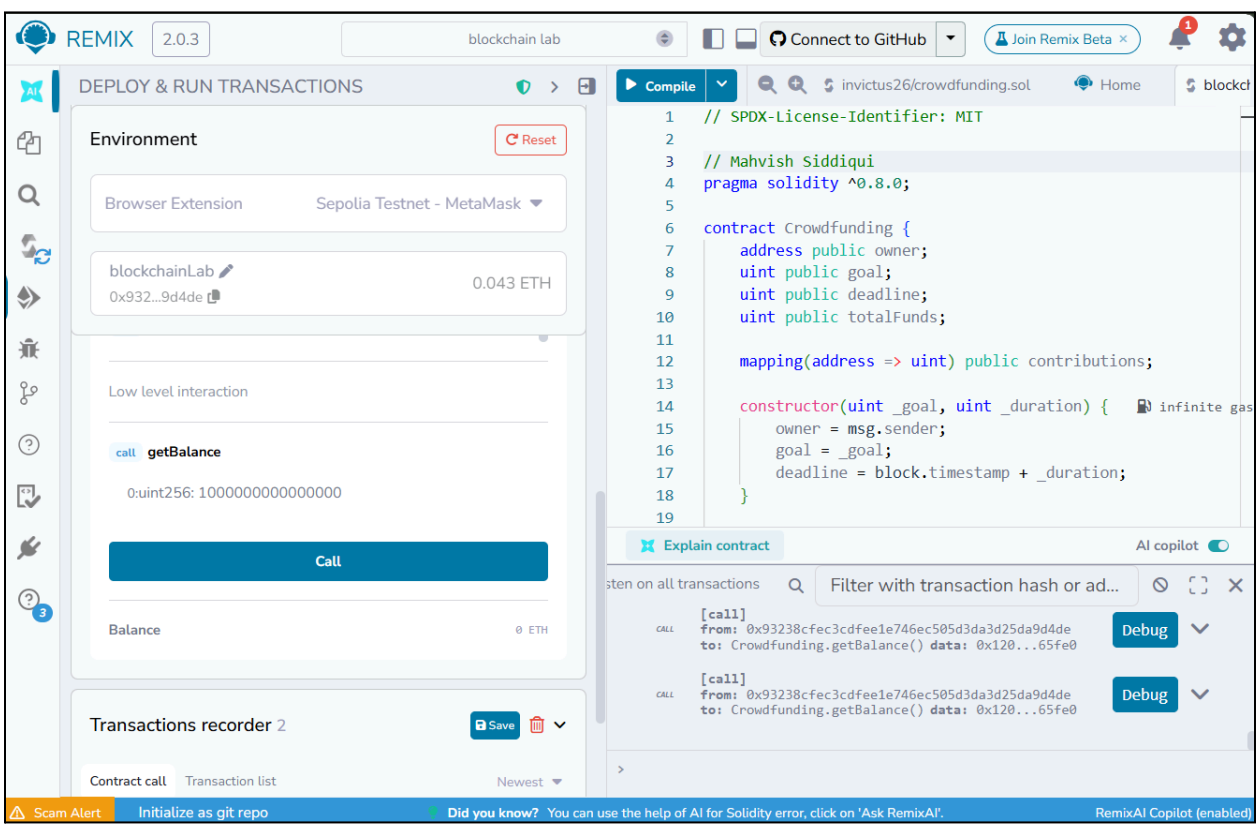

**Step 7:** copy paste the contracts address in [https://sepolia.etherscan.io/](https://sepolia.etherscan.io/) to see all transactions:



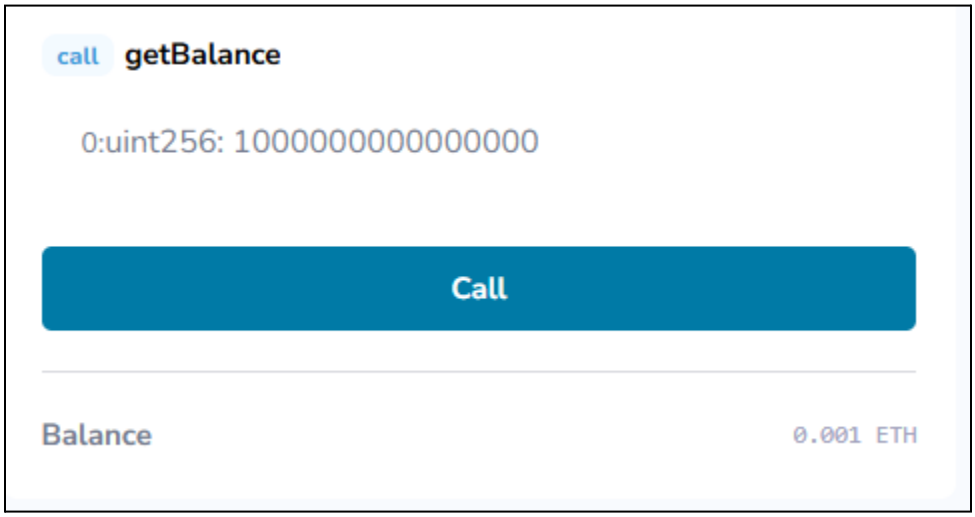
Also, we can see the transactions under all transactions in remix IDE



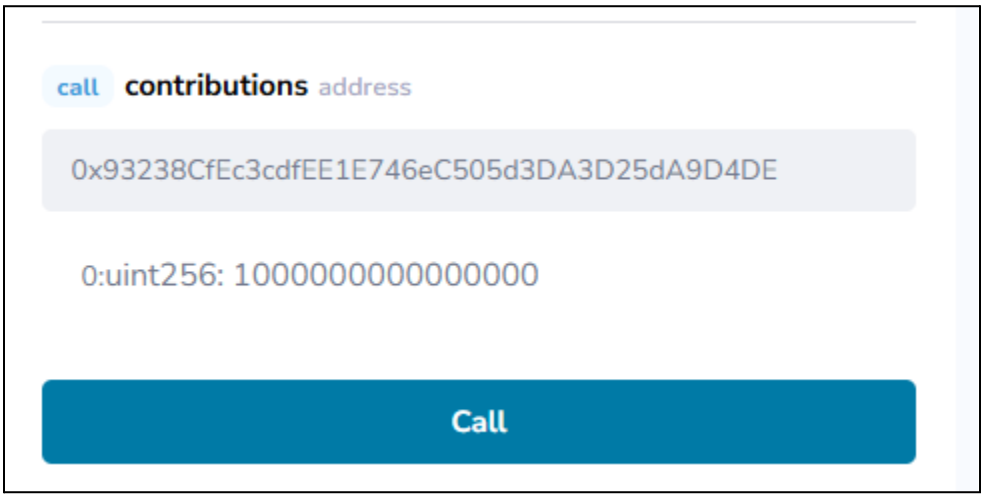

**Step 8:** use getBalance function to check the total funds received



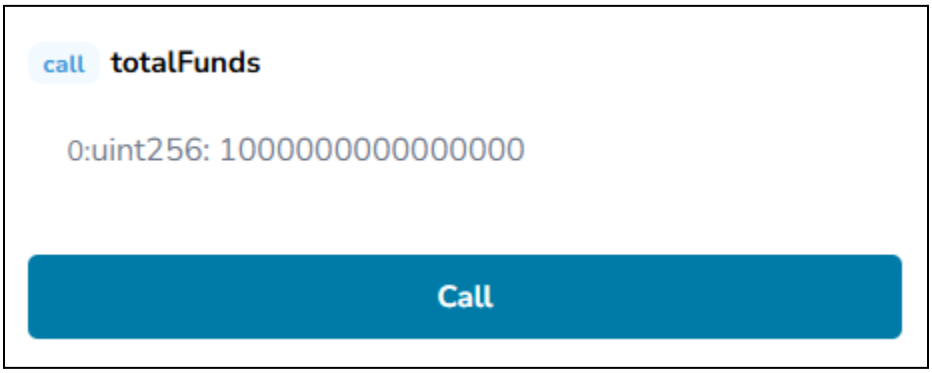
**Output:**




**Step 9:** Paste wallet address from metamask into contributions to see all transactions by the particular use




**Step 10:** use totalFunds function to see totalFunds recieved



**Conclusion:** In this experiment, the process of integrating the MetaMask wallet with the Remix IDE was understood and successfully performed. Smart contracts were deployed and transactions were executed using test networks like Sepolia Testnet. This helped in gaining practical knowledge of blockchain transactions and smart contract interaction in a safe testing environment.